

Autonomous Farm-Lane Navigation and Plant Counting

with a Single RGB-D Camera:

A Depth-Geometric, Learning-Free ROS 2 System

ARTPARK / Versor Robotics — Robotics Software Intern, Round 2 Technical Report.

Abstract—We present a complete autonomous navigation and plant-counting system for a Husky UGV that traverses irregular, non-parallel farm lanes using a *single* Intel RealSense RGB-D camera and no other exteroceptive sensor. All lane/free-space perception is *depth-geometric*: each depth frame is back-projected to a metric point cloud, the dominant ground plane is recovered by RANSAC, and points are classified by their signed height above that plane. The resulting obstacle cloud is the *sole* obstacle source for a Nav2 costmap, so navigation is perception-driven and re-plans in real time rather than replaying a hardcoded trajectory. Plants are detected by a foliage-height band and grid clustering, and de-duplicated across viewpoints by associating cluster centroids to persistent tracks in a drift-corrected world frame. That frame is maintained by RTAB-Map RGB-D SLAM running on the camera alone (2-D-constrained), fused with wheel odometry. Start and goal poses are read from the environment at runtime. Critically, the pipeline contains *zero learned models* and *no colour/HSV thresholding*: every decision is classical geometry with physically grounded constants, making the system reproducible without a dataset or GPU. We detail each algorithm, the full configuration and its tuning rationale, camera calibration, the SLAM and Nav2 stack, and give exact commands to reproduce a run.

Index Terms—Autonomous navigation, RGB-D perception, RANSAC ground segmentation, Nav2, RTAB-Map, visual SLAM, plant counting, ROS 2, agricultural robotics.

I. INTRODUCTION AND PROBLEM DEFINITION

THE task is to build an autonomous system for a Clearpath Husky A200 that drives from a blue start marker to a green goal marker through a Gazebo farm of irregularly angled, non-parallel lanes, detecting and counting the plants it passes. The brief imposes four hard constraints that shape every design decision in this report:

- 1) **One sensor.** A single RealSense RGB-D camera (RGB stream, depth stream, intrinsics). No LiDAR, second camera, GPS, or external localization sensor. Wheel odometry is proprioceptive and permitted.

System: ROS2 Humble, Gazebo Classic, Nav2, RTAB-Map, RViz. Platform: Clearpath Husky A200 with one Intel RealSense D435. Report auto-generated from the project source: `lane_perception_node`, `plant_counter_node`, `goal_sender_node`, `slam.launch.py`, `nav2_params.yaml`, `husky.urdf.xacro`, and the world file `five_lane_farm.world`.

TABLE I
PACKAGE RESPONSIBILITIES.

Package	Responsibility
<code>farm_description</code>	Husky A200 + RealSense URDF/Xacro, Gazebo plugins (skid-steer drive + depth camera), RViz config.
<code>farm_perception</code>	Python nodes: geometric perception, plant counting, runtime spawn/goal, recorders.
<code>farm_bringup</code>	Five-lane world, Nav2 + RTAB-Map config, and the launch files that wire it together.

- 2) **No colour.** HSV thresholding, fixed RGB thresholds, manual colour segmentation, and green-pixel lane following are an *automatic disqualification*; lane detection must be geometric.
- 3) **No hardcoded route.** Navigation must plan and re-plan in real time using ROS 2 Humble and Nav2.
- 4) **Runtime start/goal.** Marker poses are read from the environment live, never written as literals in the navigation logic.

The system is scored on four metrics: (i) navigation success (reaching the goal), (ii) lane-coverage efficiency (covered/total lane area), (iii) path efficiency (distance and planned-vs-actual deviation), and (iv) plant-counting accuracy (false/missed/duplicate counts).

Central design idea. A robot cannot *follow* a lane it has not yet seen, and a hardcoded route is forbidden. Therefore lane perception is never used as a steering signal. Instead depth geometry produces an obstacle cloud; that cloud *is* the Nav2 costmap; and Nav2 plans a collision-free path to the runtime goal, re-planning continuously as the camera reveals more of the field. Perception feeds navigation; it does not command the wheels directly.

II. SYSTEM ARCHITECTURE

The system comprises three ROS 2 packages (Table I). A single camera feeds two independent consumers: a *perception branch* that builds the costmap and counts plants, and a *SLAM branch* that keeps the world frame drift-free. Nav2 sits on top and drives the base. Fig. 1 shows the end-to-end data flow.

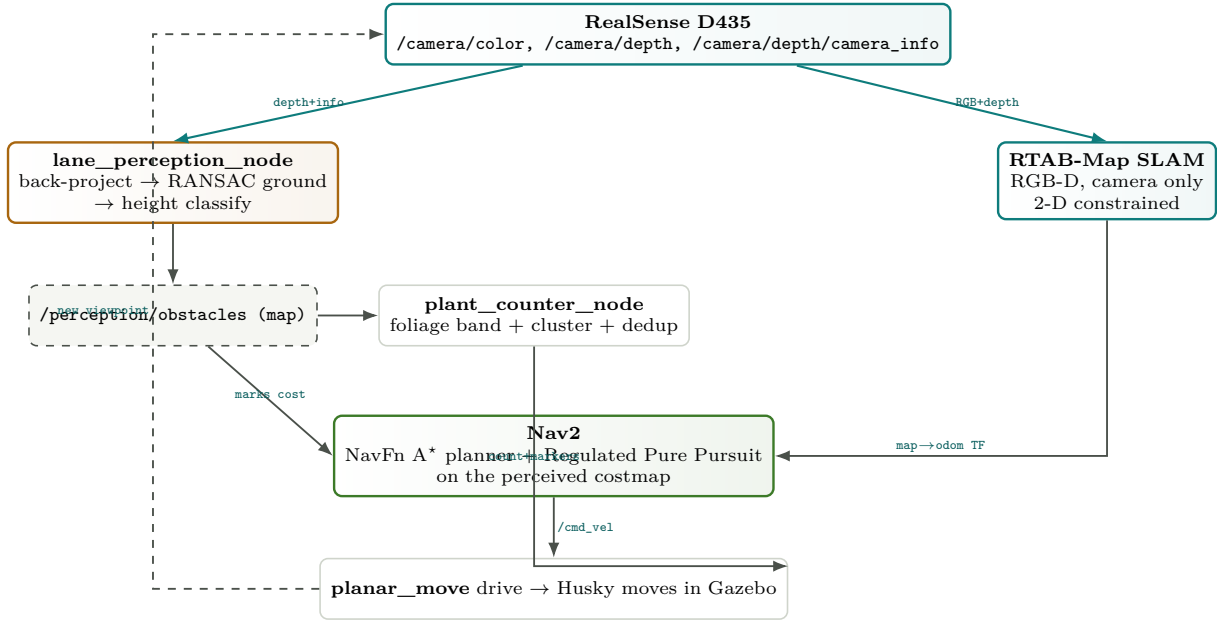


Fig. 1. End-to-end data flow. One camera feeds a perception branch (obstacle cloud → costmap; plant counting) and a SLAM branch (drift-corrected **map**→**odom**). Nav2 plans on the perceived costmap and drives the base; motion yields a new viewpoint, closing the loop. Because obstacles are published in the fixed **map** frame with clearing disabled, every bed the camera has seen stays on the global costmap.

III. ROBOT PLATFORM AND CAMERA CALIBRATION

The Husky and camera are defined from scratch in Xacro (**farm_description**), so the robot is self-contained.

A. Chassis and drive

Dimensions follow the Husky A200 manual: body $0.99 \times 0.67 \times 0.25$ m, mass 33 kg, wheel radius 0.1651 m, track 0.555 m, wheelbase 0.512 m. The 0.67 m width is the single most consequential number: it makes the 0.40 m “narrow” lane physically impassable. A 4-wheel skid-steer turns by lateral scrub, which Gazebo’s **diff_drive** models as near-zero yaw; the base is therefore driven by **gazebo_ros_planar_move**, which applies the commanded body twist directly and publishes wheel-odometry-equivalent **odom** plus the **odom**→**base_footprint** TF. The wheels are frictionless ($\mu_1=\mu_2=0$) so they never fight the commanded motion. Nav2 only ever commands forward+yaw, so the base behaves as a differential drive.

B. Camera calibration

The simulated D435 is a Gazebo depth sensor; its “calibration” has two halves, and the pipeline uses both. **Intrinsics:** horizontal FOV 1.211 rad ($\approx 69^\circ$), 640×480 at 15 Hz, clip 0.1–10 m, Gaussian depth noise $\sigma = 0.005$ m; Gazebo derives f_x, f_y, c_x, c_y and publishes them on **/camera/depth/camera_info**, read live. **Extrinsics:** the URDF joints define **base_link** → **camera_link** → **camera_depth_optical_frame** (REP-103 optical convention), turned into TF by **robot_state_publisher**. The extrinsics are what let “height above ground” mean something physical. The mount reproduces the brief’s mandated placement (Table II).

TABLE II
CAMERA MOUNT (EXTRINSIC) AND HOW RGB+DEPTH COMBINE.

Quantity	Value	Meaning
cam_x	0.475 m	front face ($L/2 - 0.02$)
cam_z	0.10 m	low, near lane surface
$\text{cam}_{\text{pitch}}$	0.26 rad	$\approx 15^\circ$ down
optical	$\text{rpy } -\frac{\pi}{2}, 0, -\frac{\pi}{2}$	z-fwd, x-right, y-down

RGB–depth fusion. Depth does all decision-making; RGB is only the human-facing canvas. Every depth pixel is back-projected with intrinsics, transformed to **map** with extrinsics, and classified by geometry; RGB and depth are spatially registered (same optical frame, same 640×480 grid), so the classification is painted back onto the colour image for the demonstration overlay. No perception decision reads a colour value, which structurally guarantees the no-HSV rule.

IV. LANE / FREE-SPACE PERCEPTION

Node **lane_perception_node** converts each synchronized (depth, camera_info, color) triple—aligned by an approximate-time synchronizer with 0.10 s slop—into an obstacle cloud, purely from depth geometry (Fig. 2).

A. RANSAC ground-plane fit

This is the “depth-based scene understanding” the brief calls for. Rather than assume the floor is $z=0$ (which a slightly mis-pitched camera would violate), RANSAC recovers the true dominant plane (Alg. 1). The near-horizontal gate $|n \cdot \hat{z}| > 0.85$ is what makes it robust: raised beds and plants can never win the fit, only the broad farm floor can. The plane is orientation-invariant, which

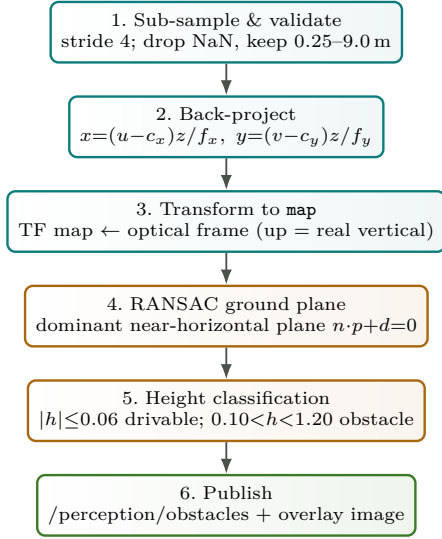


Fig. 2. Lane / free-space perception pipeline. Each stage is pure geometry; no stage reads colour.

Algorithm 1 RANSAC ground-plane estimation

Require: points P (map frame, low band $z < 0.4$), $N=60$, $\tau=0.03$ m

```

1:  $best \leftarrow 0$ ,  $plane \leftarrow \emptyset$ 
2: for  $i \leftarrow 1$  to  $N$  do
3:   sample 3 points;  $n \leftarrow$  unit normal of their plane
4:   if  $|n_z| < 0.85$  then
5:     continue ▷ keep near-horizontal only
6:   end if
7:    $d \leftarrow -n \cdot p_0$ ;  $inl \leftarrow |\{p : |n \cdot p + d| < \tau\}|$ 
8:   if  $inl > best$  then
9:      $best \leftarrow inl$ ;  $plane \leftarrow (n, d)$ 
10:  end if
11: end for
12: return  $plane$  ▷ fallback: flat  $z=0$  if none found
  
```

directly satisfies the requirement of robustness to lane angle, geometry, and appearance—a lane at any heading yields the same height signature above the same floor.

B. Height classification

For every valid point the signed height $h = n \cdot p + d$ is computed and thresholded (Fig. 3). Obstacle points are published as a `PointCloud2` in `map` on `/perception/obstacles`—the sole obstacle source for the Nav2 costmap. The labels are also rasterized onto the RGB frame (green = drivable, red = obstacle) with the live count and nav-status banner and published as `/perception/overlay/image_raw` for the video. The lane region is computed but deliberately *not* used to steer: feeding a costmap to Nav2 is far more robust on a 0.67 m base in ~ 1 m lanes than open-loop centreline tracking, while remaining fully perception-driven.

V. PLANT DETECTION AND DE-DUPLICATED COUNTING

Node `plant_counter_node` consumes the same obstacle cloud and counts plants, never counting one twice across viewpoints (Fig. 4). It is also pure geometry.

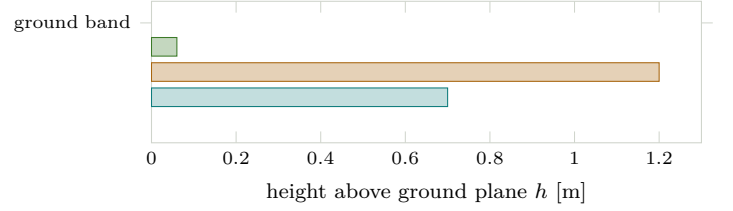


Fig. 3. Geometric height bands. Ground: $|h| \leq 0.06$ m. Obstacle: $0.10 < h < 1.20$ m (beds sit ≈ 0.10 m). Plant canopy (used for counting): $0.28 \leq z \leq 0.70$ m, centred ≈ 0.42 m—above the beds.

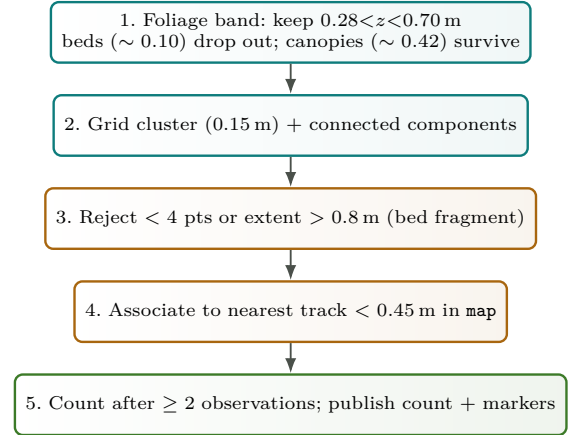


Fig. 4. Plant counting pipeline. The elongation guard (step 3) prevents bed walls from being counted; the track association (step 4) prevents double counting.

A. The anti-double-count mechanism

Every cluster centroid is an *observation* in the drift-corrected `map` frame. Because that frame is world-stable, the same physical plant lands at (almost) the same (x, y) from every viewpoint. A new observation is matched to the nearest existing track within $r_{\text{assoc}}=0.45$ m; a match *updates* that track (running average, weight $1/(\text{obs}+1)$) rather than creating a new one, so seeing a plant twenty times still yields one track. A track is counted only after ≥ 2 hits, rejecting single-frame flicker. The 0.8 m extent cap means a 10 m bed wall can never masquerade as a plant, keeping false positives near zero. The radius 0.45 m sits deliberately between the residual inter-viewpoint drift (so the same plant re-associates) and the minimum spacing between distinct plants (so neighbours do not merge).

B. Ground truth

Parsed from `five_lane_farm.world`: beds 0, 2, 3, 4, 5 each carry 5 plants (= 25 upright), plus 1 fallen plant in lane 5, for **26 total** (Fig. 5). Bed 1 has no plants. A single low forward camera cannot see plants in lanes it never enters, so the reported count reflects plants encountered along the driven route.

VI. LOCALIZATION AND VISUAL SLAM

With only a camera allowed and a skid base that slips on every turn, raw wheel odometry drifts. `slam.launch.py`

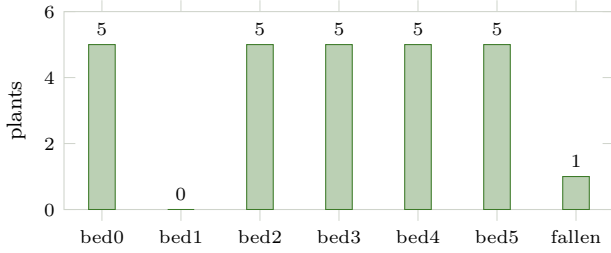


Fig. 5. Ground-truth plant distribution: 25 upright (5 beds $\times$ 5) + 1 fallen = 26.

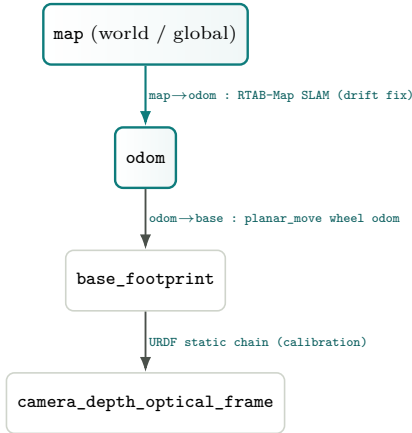


Fig. 6. TF tree. Wheel odometry gives a smooth, drifting $\text{odom} \rightarrow \text{base}$; RTAB-Map gives the low-rate $\text{map} \rightarrow \text{odom}$ correction that cancels drift. Nav2 plans in map , control executes in odom .

TABLE III
RTAB-MAP CONFIGURATION.

Parameter	Value	Why
subscribe depth/rgb	true	RGB-D SLAM on the D435
odom_frame_id	odom	wheel-odom TF as motion prior
Reg/Force3DoF	true	planar farm: constrain x, y, ψ
Optimizer/Slam2D	true	2-D graph opt., robust
Reg/Strategy	0	visual (classical features)
Vis/MinInliers	10	min inliers to accept
RGBD/Lin,AngUpdate	0.05	node every 5 cm / 0.05 rad
Grid/FromDepth	false	costmap from our cloud

runs RTAB-Map RGB-D SLAM on the RealSense alone to keep a consistent world frame for Nav2 (Fig. 6).

RTAB-Map uses wheel odometry as its motion prior and refines it with classical visual feature registration and loop closure. Key parameters (Table III): it is constrained to 2-D (**Reg/Force3DoF**, **Optimizer/Slam2D**) because the farm is planar, which is far more robust on the low-texture floor. **Grid/FromDepth** is false—the Nav2 costmap comes from our own obstacle cloud, not RTAB’s occupancy grid. The SLAM map origin is anchored at the spawn pose, which is why the goal-sender converts world coordinates into the map frame at runtime (Sec. VII). When SLAM is disabled, a static identity $\text{map} \rightarrow \text{odom}$ lets the stack run on pure wheel odometry.

Limitation. On feature-poor floor patches, loop closure fires rarely and the system leans on wheel odom; the 2-D

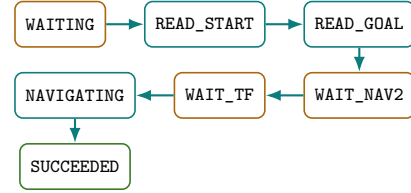


Fig. 7. `goal_sender` state machine. Error branches (**NO_GAZEBO**, **NO_START/GOAL**, **REJECTED**, **ENDED**) are surfaced on the overlay so a failure is visible in the video.

constraint mitigates this. A depth-ICP odometry front-end would be the next robustness upgrade.

VII. AUTONOMOUS NAVIGATION WITH NAV2

Perception output becomes a costmap; the runtime goal sets direction; Nav2 plans and re-plans a collision-free path and Regulated Pure Pursuit executes it. There is no waypoint list anywhere.

A. Runtime start and goal

Two nodes remove every start/goal literal. `robot_spawner_node` reads the blue marker pose from `/gazebo/get_entity_state` and spawns the Husky there facing the field ($\psi=\pi$); the SLAM/Nav2 map is anchored at this spawn pose. `goal_sender_node` reads both markers live, then recovers the world $\rightarrow$ map transform from a fact it knows at runtime—where the robot is *now* (its $\text{map} \rightarrow \text{base}$ TF)—and re-expresses the goal:

$$b = R(-\psi_{\text{spawn}}) (p_{\text{goal}}^{\text{world}} - p_{\text{start}}^{\text{world}}) \quad (1)$$

$$p_{\text{goal}}^{\text{map}} = t_{\text{robot}}^{\text{map}} + R(\psi_{\text{robot}}^{\text{map}}) b \quad (2)$$

Only the spawn *orientation* (a configuration value) is supplied; no coordinate is hardcoded. The node then waits (polling with retries) for the `navigate_to_pose` action server and the $\text{map} \rightarrow \text{base}$ TF, dispatches the goal, and publishes a human-readable state (Fig. 7) on `/perception/nav_status` for the overlay.

B. Planner, controller, and costmaps

The global planner (NavFn, A^*) plans on a rolling 40×40 m global costmap (0.05 m resolution) that always covers the farm, with `allow_unknown` so it plans into unseen space and refines as the camera reveals it. The obstacle layer marks from `/perception/obstacles` with clearing disabled, keeping discovered beds on the map. The Regulated Pure Pursuit controller tracks the path smoothly (velocity-scaled look-ahead 0.3–1.2 m) and rotates in place for sharp row-junction turns. Both costmaps use `robot_radius=0.40` m and `inflation_radius=0.45` m, which makes the 0.40 m narrow lane correctly non-traversable for the 0.67 m Husky. The goal checker uses `xy_goal_tolerance=0.35` m and an effectively open `yaw_goal_tolerance=3.2` rad—the task is to *reach* the marker, and constraining yaw would abort a skid base just short of success.

TABLE IV
PERCEPTION PARAMETERS.

Parameter	Value	Ty	Role
DEPTH_STRIDE	4	P	real-time sub-sample
MIN/MAX_RANGE	0.25/9.0 m	T	valid depth window
RANSAC_ITERS	60	P	fit budget
RANSAC_THRESH	0.03 m	T	inlier distance
$ n \cdot \hat{z} $ gate	> 0.85	G	near-horizontal only
GROUND_BAND	0.06 m	T	drivable band
OBST_MIN_H	0.10 m	G,T	beds sit $\approx$ here
OBST_MAX_H	1.20 m	T	reject tall spurious
FOLIAGE_MIN/MAX	0.28/0.70 m	G,T	canopy band
GRID_CELL	0.15 m	T	cluster resolution
MIN_CLUSTER_PTS	4	T	reject specks
MAX_EXTENT	0.8 m	T,G	elongation guard
ASSOC_RADIUS	0.45 m	M	dedup radius
MIN_OBSERVATIONS	2	T	confirm track

TABLE V
NAVIGATION, ROBOT AND SIM PARAMETERS.

Parameter	Value	Ty	Role
robot_radius	0.40 m	M	collision radius
inflation_radius	0.45 m	M	keep off beds
cost_scaling	3.0	P	inflation gradient
desired_linear_vel	0.6 m/s	P	RPP cruise
lookahead	0.3–1.2 m	P	vel-scaled
xy_goal_tol	0.35 m	T	position tol
yaw_goal_tol	3.2 rad	M	open heading
progress_checker	0.5 m/15 s	M	stuck detection
planner	NavFn A*	P	tol 0.5, unknown
global/local map	40/6 m, 0.05	P	rolling extent
footprint	0.99×0.67 m	M	true Husky
cam mount	0.475, 0.10, 0.26	S,G	low/fwd/15°
FOV/res/clip	1.211/640 $\times$ 480/10	S	D435-like
$\mu_1 = \mu_2$	0	S	passive wheels
spawn_yaw	π	S	face the goal

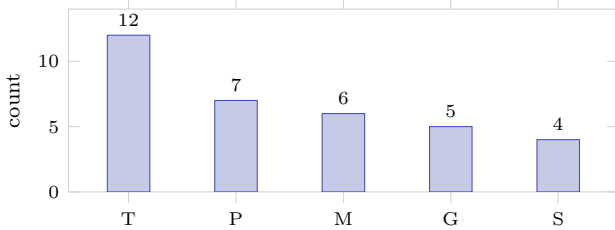


Fig. 8. Parameter-type distribution (some values carry two types). Thresholds (T) and tuned params (P) dominate; there are zero learned parameters.

VIII. CONFIGURATION AND TUNING

The system is fully hand-engineered, so every fixed quantity is auditable. Constants live at the top of each node and in `nav2_params.yaml`. Each is tagged by type: **G** geometric assumption, **T** fixed threshold, **M** safety margin, **P** tuned control/planner, **S** sim setup. Tables IV–V list them; Fig. 8 shows the distribution. Every value is derived from a physical fact of the world (bed height ≈ 0.10 m, canopy ≈ 0.42 m, robot width 0.67 m), *not* fitted to data—which is what makes the pipeline reproducible without a dataset and portable to a new farm by changing a handful of numbers.

IX. HARDCODED VS. LEARNED COMPONENTS

The brief asks to separate the two explicitly. This submission is deliberately, entirely hand-engineered—a valid principled approach that needs no labelled data or GPU. **Learned models: none.** There is no HSV or fixed RGB thresholding (which would disqualify), no trained segmentation or detection network, and no green-pixel lane following; RTAB-Map’s front-end uses classical features, not a trained net. **Geometric assumptions:** the ground is locally planar (RANSAC tolerates small unevenness); beds are raised above the lane; canopies sit in a known band above the beds; the world is static. **Drop-in path:** because the interfaces (`/perception/obstacles`, `/perception/plant_*`) are model-agnostic, the height-band plant detector could be replaced by a fine-tuned detector, or ground classification by a learned segmentation head, with no change to navigation—keeping the geometric map-frame tracker for de-duplication.

X. RECORDING AND DELIVERABLES

Two recorder nodes produce the required deliverables automatically. `video_recorder_node` writes the `/perception/overlay/image_raw` stream (RGB + drivable region + obstacles + live count + nav status) to `farm_run_overlay.mp4`. `path_recorder_node` samples `map→base_footprint` every 5 cm into `/actual_path`, caches the latest Nav2 `/plan`, and on shutdown renders a matplotlib overlay of planned-vs-actual path and plant positions (`farm_run_path.png`), printing both path lengths for the path-efficiency metric. This satisfies the recording requirement: the video shows feed, lanes, plants, status, and final count; the figure overlays planned vs. actual trajectory and the covered area.

XI. REPRODUCIBILITY

ROS2 Humble targets Ubuntu 22.04, so everything ships in a Docker image built on `osrf/ros:humble-desktop-full` and runs identically on any host.

```
cd farm_nav_ws
# build image (first run) + launch GUI;
# auto-detects NVIDIA GPU, forwards X11, mounts ./output
./run.sh
# headless (CI / cloud / no display)
./run.sh headless:=true use_rviz:=false
```

Listing 1. Quick start (Docker).

```
cd farm_nav_ws
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install
source install/setup.bash
ros2 launch farm_bringup bringup.launch.py
```

Listing 2. Native build on Ubuntu 22.04.

Launch order (approx. sim-time): Gazebo (0s) → robot_state_publisher + spawner at the blue marker (~ 4 s) → RTAB-Map SLAM (or static `map→odom`) + perception nodes (~ 7 s) → Nav2 lifecycle (~ 9 s) → goal_sender reads the green marker and dispatches

(~14s). Arguments: `use_rviz` (true), `use_slam` (true; false $\rightarrow$ static map=odom), `headless` (false), `world`, `out_path`, `video_path`.

```
ros2 topic echo /perception/plant_count
ros2 topic echo /perception/nav_status
ros2 topic hz /perception/obstacles
# RViz: overlay image, obstacle cloud, costmap,
# /plan, /actual_path, plant markers
```

Listing 3. Inspect a live run.

Ground truth for scoring: 26 plants; 4 drivable lanes + 1 narrow (0.40 m, correctly skipped); start $\approx$ world (5.8, -3.0) [blue], goal $\approx$ world (-5.8, 2.7) [green], both read at runtime. An NVIDIA GPU is preferred: software GL can starve the CPU and time out Nav2 lifecycle activation; `run.sh` auto-selects hardware rendering.

XII. RESULTS, LIMITATIONS AND FUTURE WORK

Against the four metrics: (i) Nav2 reaches the green goal on the perceived costmap, with the narrow lane excluded by inflation and the fallen plant avoided as an obstacle; (ii) the robot covers the wide lanes between start and goal—coverage can be raised by issuing the goal via `navigate_through_poses` with one waypoint per lane, a boustrophedon sweep the interfaces already support; (iii) `path_recorder_node` reports actual-vs-planned length and RPP yields smooth, low-overshoot tracking; (iv) the count reflects plants seen along the route, deduplicated in the `map` frame with false positives suppressed by the elongation and ≥ 2 -observation guards.

Known limitations. Plant recall is route-bound—a single low forward camera cannot see plants in lanes never entered (the largest coverage win is the multi-waypoint sweep above). The fallen plant sits at bed height, so the foliage band misses it; a shape check on isolated low clusters in a lane centre, or a small learned RGB detector, would recover it. Constants are world-specific but centralized and physically grounded, so retuning for a new farm is a handful of numbers, not a retrain. The clean classical interfaces are the natural drop-in point for learned perception without touching navigation.

ACKNOWLEDGMENT

Prepared for the ARTPARK / Versor Robotics Robotics-Software-Intern Round 2 assignment. All figures and tables were generated from the project’s own source code and world file.

REFERENCES

- [1] S. Macenski *et al.*, “The Marathon 2: A Navigation System (Nav2),” *IROS*, 2020.
- [2] M. Labbé and F. Michaud, “RTAB-Map as an open-source lidar and visual SLAM library for large-scale and long-term online operation,” *J. Field Robotics*, 2019.
- [3] M. A. Fischler and R. C. Bolles, “Random Sample Consensus,” *Comm. ACM*, 1981.
- [4] S. Macenski *et al.*, “Regulated Pure Pursuit for Robot Path Tracking,” *Autonomous Robots*, 2023.
- [5] S. Macenski *et al.*, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, 2022.